Problem A. Bus Analysis

Divide all prices by 2, so the two ticket prices become 1 and 3. Multiply the final answer by 2.

First consider how to compute the minimum cost for fixed times $t_1, t_2, \ldots, t_n$. Let $\mathrm{dp}(i, j)$ be the maximum remaining distance that can still be covered after spending cost j to cover the first i points. If the minimum cost for covering the first i points is x, then only

$$f(i, x), \quad f(i, x+1), \quad f(i, x+2)$$

are useful. Spending at least $x + 3$ is never better than spending x and then buying one ticket of cost 3.

Thus the DP process only needs to remember x and the three values $f(i, x), f(i, x+1), f(i, x+2)$. For counting, store these three values in the state. Since only the sum of costs is needed, x itself does not need to be part of the state; whenever the minimum cost increases, add its contribution to the answer.

The time complexity is $O(n \cdot 75^3)$. In practice the constant is small: useful triples (a, b, c) satisfy

$$0 \le a \le a + 20 \le b \le b + 20 \le c \le 75.$$

Problem B. Missing Boundaries

First, intervals whose two endpoints are already fixed must not intersect.

For intervals with exactly one fixed endpoint, the fixed endpoint must not lie inside an already fixed interval, and all such fixed endpoints must be distinct.

All remaining intervals can be filled arbitrarily. Therefore, compute the minimum and maximum possible number of intervals and check whether the required number lies within this range.

The maximum number of intervals is

$$L - (\text{total length of intervals with two fixed endpoints}) - (\text{number of intervals with one fixed endpoint}).$$

The minimum number of intervals is the number of adjacent pairs consisting of a right endpoint followed by a left endpoint whose gap is greater than 1, including the beginning and the end.

The time complexity is $O(N \log N)$.

Problem C. Price Combo

Consider the final decision of whether each item is bought in shop A or shop B. If $a_i > a_j$ and $b_i < b_j$, then buying item i in shop A and item j in shop B is never optimal, because swapping these two choices cannot increase the cost.

Thus, after viewing every item as a point (a_i, b_i), there is a monotone staircase from the upper right to the lower left: points on the staircase and to its upper-left side are bought in shop A, while points to the lower-right side are bought in shop B. We perform DP over this staircase.

Assume first that all a_i and b_i are distinct. Traverse the staircase from upper right to lower left. When the staircase moves left, it adds the contribution of a_i for points above the staircase. When it moves down, it adds the contribution of b_i for points to the right of the staircase. Because of the buy-one-get-one rule, only the 1st, 3rd, 5th, ... chosen item contributes.

It is enough to use the points (a_i, b_i) as critical points, and for convenience add $(+\infty, +\infty)$ and $(-\infty, -\infty)$. Let

$$\mathrm{dp}(i, p, q)$$

be the minimum cost when the staircase is at (a_i, b_i), the parity of the number of selected a-items is p, and the parity of the number of selected b-items is q.

Suppose (a_i, b_i) is reached from (a_j, b_j), where $a_i < a_j$ and $b_i < b_j$. The added contribution comes from the points satisfying

$$a_i < a_k < a_j, \quad b_k > b_j,$$

and the points satisfying

$$b_i < b_k < b_j, \quad a_k > a_i,$$

together with the contribution of a_i itself.

To compute the DP efficiently, sweep in decreasing order of a_i and maintain a segment tree indexed by b. The tree stores DP values plus the accumulated a-contributions. For a_k contributions, do prefix additions on the range $b_j < b_k$. The lazy tag is a triple: the value added to parity 0, the value added to parity 1, and the number of added items. The b_k contributions are maintained directly in the segment tree: each node stores the minimum of the DP value plus the contribution of smaller b values, as well as the contribution of its own b values. When merging, the DP value is the minimum of the left interval value and the right interval value plus the left interval's b-contribution.

Then $\mathrm{dp}(i)$ is obtained by querying the suffix $(b_i, +\infty)$.

The time complexity is $O(N \log N)$.

Problem D. Data Determination

Assume $a_1 \le a_2 \le \cdots \le a_n$.

If k is odd, we need an index i such that

$$a_i = m, \qquad i - 1 \ge \frac{k-1}{2}, \qquad n - i \ge \frac{k-1}{2}.$$

If k is even, we need indices $i < j$ such that

$$a_i + a_j = 2m, \qquad i - 1 \ge \frac{k}{2} - 1, \qquad n - j \ge \frac{k}{2} - 1.$$

For a fixed pair of values (a_i, a_j), it is always best to choose i and j as close as possible. Therefore there are only $O(n)$ relevant pairs to check.

The time complexity is $O(n \log n)$.

Problem E. Express Eviction

Solution 1: Minimum cut

There exists a feasible path if and only if there is no blocking path from the lower-left region to the upper-right region such that every cell on it is occupied and consecutive cells differ by at most 2 in both row and column. We regard the lower-left boundary as adjacent to the lower-left region, and the upper-right boundary as adjacent to the upper-right region.

For every occupied cell u, create two vertices a_u and b_u. If u touches the lower-left boundary, add an edge $(S, a_u, +\infty)$. If it touches the upper-right boundary, add $(b_u, T, +\infty)$. Add an edge $(a_u, b_u, 1)$, meaning we may delete this occupied cell at cost 1. For every pair of occupied cells u, v whose row and column differences are both at most 2, add $(b_u, a_v, +\infty)$.

The minimum cut in this graph is the answer.

The time complexity is $O(\mathrm{Maxflow}(HW, HW))$.

Solution 2: Shortest path

Consider when an ordinary shortest-path formulation fails. For example:

```
.#....
....#.
...#..
###...
###..#
###...
```

In this grid, it is enough to delete the fourth cell in the third row. The reason for the failure is that the path can pass through two opposite corners of the same cell, but the state does not remember that cell, so its deletion cost may be counted twice.

Consider all cells that are visited twice by such a path. If an occupied cell u is visited twice inside the two visits of another cell v, then it is no worse to pay one extra unit and pass directly through v. Hence we may assume that no such nested pair (u, v) exists. Under this assumption, each point on the path lies between at most two such pairs of visits, one from each side of the path. Recording these two cells in the state gives an $O(H^3W^3)$ algorithm.

This can be improved. Suppose the path visits cells u and v in this order. Then the next step must go from v to the corner diagonally opposite to u. Moreover, if any extra cost is paid before returning to u, it is no worse to pass directly through u. Therefore, it is enough to remember only one previously visited cell.

Add one extra transition: if the current position is x and the remembered cell is v, then for every point w diagonally adjacent to v, if x can reach w without extra cost, move to position w and remember x. These reachability transitions can be precomputed by BFS from every point.

The time complexity becomes $O(H^2W^2)$.

Problem F. Fibonacci Fusion

Assume $a_1 \le a_2 \le \cdots \le a_n$.

Enumerate j. Then a Fibonacci value of the form $a_i + a_j$ must satisfy

$$a_j < a_i + a_j \le 2a_j.$$

There are at most two Fibonacci numbers in this range. If we can find those two candidates, it remains only to count how often a certain value occurs among the a_i.

The values a_i may be very large, so the corresponding Fibonacci numbers may also be huge. Compute Fibonacci numbers modulo large primes to avoid collisions, and estimate the index using

$$F_n = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right) \approx \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^n.$$

Thus

$$n \approx \log_{(1+\sqrt{5})/2}(\sqrt{5}\, a_i).$$

The time complexity is $O(n \log n + \sum \log a_i)$.

Problem G. Game of Geniuses

The answer is

$$\max_i \min_j a_{ij}.$$

Let this value be x. The lower bound is straightforward: the first player can delete all rows except a row whose minimum is x.

For the upper bound, it is enough to show that after each round, the second player can guarantee that every remaining row contains a number at most x. Suppose the first player deletes the first row. In each of

the remaining $n-1$ rows, the second player chooses one entry at most x, marks its column, and deletes every unmarked column.

The time complexity is $O(n^2)$.

Problem H. Henry the Plumber

Every pipe segment must turn immediately, so the answer is at least 2.

The answer is at most 4: connect each endpoint by a pipe parallel to the z-axis to a common z value, then connect them in a plane perpendicular to the z-axis. If the endpoints already differ only in z, connect them directly.

Now split into cases.

- If the vector $(x_1, y_1, z_1) - (x_2, y_2, z_2)$ is perpendicular to both $(p_1, q_1, 0)$ and $(p_2, q_2, 0)$, the answer is 2.
- Otherwise the answer is at least 3. It is 3 if and only if there exists an intermediate point P such that the segment from (x_i, y_i, z_i) to P is perpendicular to $(p_i, q_i, 0)$ for both $i = 1, 2$. Thus P must lie on two planes.
- If these two planes are parallel, the answer is 4.
- Otherwise their intersection is a line parallel to the z-axis; write it as (x_0, y_0, z). We must choose z so that the angle at P is a right angle, i.e.

$$(x_0 - x_1, y_0 - y_1, z - z_1) \cdot (x_0 - x_2, y_0 - y_2, z - z_2) = 0.$$

- If (x_0, y_0) equals one endpoint's (x_i, y_i), then the candidate point is (x_i, y_i, z_j). If $z_1 \neq z_2$, the answer is 3; otherwise it is 4, because a pipe of length 0 is not allowed.
- Otherwise, the condition is a quadratic equation in z. If its discriminant is nonnegative, the answer is 3; otherwise it is 4.

The time complexity is $O(1)$.

Problem I. ICPC Inference

Enumerate the teams that receive gold, silver, and bronze. The bronze team must have solved exactly one problem, with penalty as large as possible. The gold team must have solved as many problems as possible, and among those, have penalty as small as possible.

Represent a result by the score

$$M \cdot \text{num} - \text{penalty},$$

where M is sufficiently large. A larger score means a better rank.

Consider the possible score of the silver team. Since bronze solved only one problem, the silver score should be as small as possible subject to being at least the bronze score. Therefore the only useful scores are scores with one solved problem, and scores with two solved problems and maximum penalty.

Let these possible scores be

$$s_1 < s_2 < \cdots < s_k.$$

Then any gold-bronze pair satisfying

$$s_i < \text{bronze} \leq \text{gold} < s_{i+1}$$

is invalid. The ranges below s_1 and above s_k are handled similarly.

Each interval can be processed in $O(\log N)$ time, but there may be $O(N^2)$ intervals. The penalty constant is $K = 20$, which is small. All possible penalties have the form $t_i + Kc$ with $0 \leq c < i$, so the possible penalties can be decomposed into $O(N)$ arithmetic segments of period K.

For intervals crossing different segments, answer in $O(\log N)$ time. For intervals inside one segment, reduce them to $O(K)$ queries of the form:

- count pairs (bronze, gold) such that bronze mod $K = i$, $l \le$ bronze $\le r$, and gold $-$ bronze $\le j$.

There are only $O(NK)$ relevant pairs, so after preprocessing these queries can be answered in $O(\log N)$ time.

Finally, gold, silver, and bronze must be three distinct teams, so apply inclusion-exclusion for coincidences.

The time complexity is $O(NK \log N)$.

Problem J. Juliet Unifies Ones

The constraints are small, so many approaches work.

A linear DP is to divide the final string into three consecutive parts of the form $0, 1, 0$. Let $\mathrm{dp}(i, j)$ be the minimum number of deleted characters after considering the first i characters and currently being in the j-th part.

Problem K. Routing K-Codes

If

$$K(b) = \left\lfloor \frac{K(a)}{2} \right\rfloor,$$

orient the edge as $a \to b$. Since all values $K(x)$ are distinct, every vertex has outdegree at most 1 and indegree at most 2, and the graph has no directed cycle. Thus the graph must be an inward binary tree. If $m \neq n - 1$, there is no solution.

Assume the root is fixed. Its value is 0 or 1; if its degree is less than 2, it is 0, and if its degree is 2, it is 1. The depth is at most 32 or 31. Each child of a vertex with value $K(x)$ has value either $2K(x)$ or $2K(x) + 1$.

The minimum cost sum for a fixed root can be computed by DP. Then rerooting DP gives the answer for every possible root.

The time complexity is $O(n + m)$.

Problem L. Random Numbers

The expected sum of an interval of length k is

$$k \cdot \frac{n+1}{2}.$$

Therefore a valid interval is likely only when k is close to $n/2$. When k is small, the total number of possible intervals is also small, so brute force is acceptable.

Choose a constant B. Check all intervals whose length belongs to

$$[1, B] \quad \text{or} \quad \left[\frac{n}{2} - B, \frac{n}{2} + B\right].$$

The time complexity is $O(nB)$. For example, $B = 500$ is enough to pass.

Problem M. Mathematics Championships

The answer is the maximum, over all $0 \le i \le k$, of the sum of the largest 2^i numbers. After sorting the numbers, compute these prefix sums and take the maximum.

The time complexity is $O(n2^n)$.